

Exploring Reset Types — Foundation for RDC

Introduction

In digital system design, various reset schemes are used to initialize logic, control state, and recover operation after faults or during system startup. Understanding asynchronous reset, synchronous reset, global reset, and domain-specific reset is essential for robust integrated circuit and FPGA development. This lab introduces each reset style in RTL, guides simulation with Verilator, and shows how to analyze their distinct behaviors in GTKWave.

Various types of reset are :

- **Asynchronous resets** act immediately on the target flip-flop, regardless of the clock, and are often used for fast or external initializations.
- **Synchronous resets** only affect the logic at the next relevant clock edge, providing predictable timing and synthesis.
- **Global reset** can initialize many blocks at once, typically at system start.
- **Domain-specific resets** allow for isolated control of different clock or logic regions within a design.

Procedure to create a lab:

Create lab directory

1. Create a lab directory for all types of resets: `mkdir reset_lab`
2. Navigate to lab directory : `cd reset_lab`
3. Again create lab directories for `asynchronous_reset`, `synchronous_reset`, `global_reset`, `domain_specific_reset` respectively inside `reset_lab` directory

Lab - 1: Asynchronous Reset

Objective of this lab is to understand how a flip-flop resets immediately on the negative edge of an asynchronous reset signal.

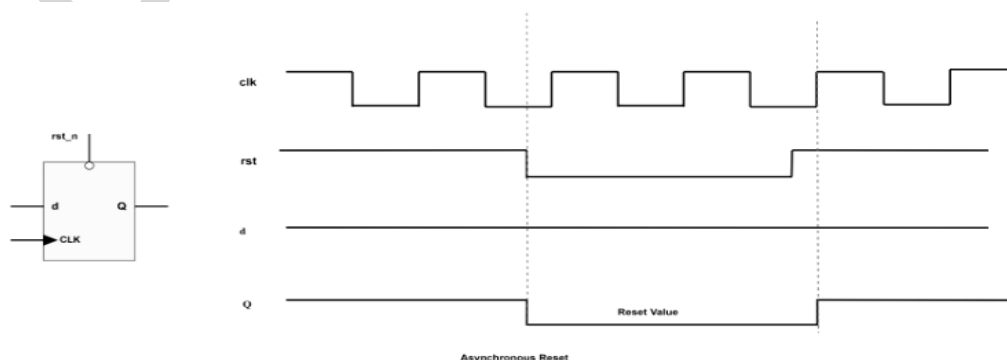


Fig: Asynchronous Reset

1.1 Navigate to asynchronous_reset directory

1.2 Create the files for asynchronous_reset : async_reset_ff.v

```
module async_reset_ff (  
    input clk,          // Clock input  
    input rst_n,        // Active-low asynchronous reset  
    input d,            // Input data  
    output reg q        // Output  
);  
  
// Reset has priority and is sampled on negative edge  
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n)  
        q <= 0;  
    else  
        q <= d;  
    end  
endmodule
```

1.3 Create testbench for asynchronous reset: async_reset_tb.v

```
module async_reset_tb;  
    reg clk = 0, rst_n = 1, d = 0;  
    wire q;  
    async_reset_ff uut (.clk(clk), .rst_n(rst_n), .d(d), .q(q));  
    always #5 clk = ~clk;  
    initial begin  
        $dumpfile("async_reset.vcd");  
        $dumpvars(0, async_reset_tb);  
        d = 1; #8;  
        rst_n = 0;    // Trigger async reset mid-cycle  
        #4;  
        rst_n = 1;  
        #20;  
    end  
endmodule
```

```

        $finish;

    end

endmodule
    
```

1.4 : Simulation Script

a) Create “run.sh”

```

#!/bin/bash

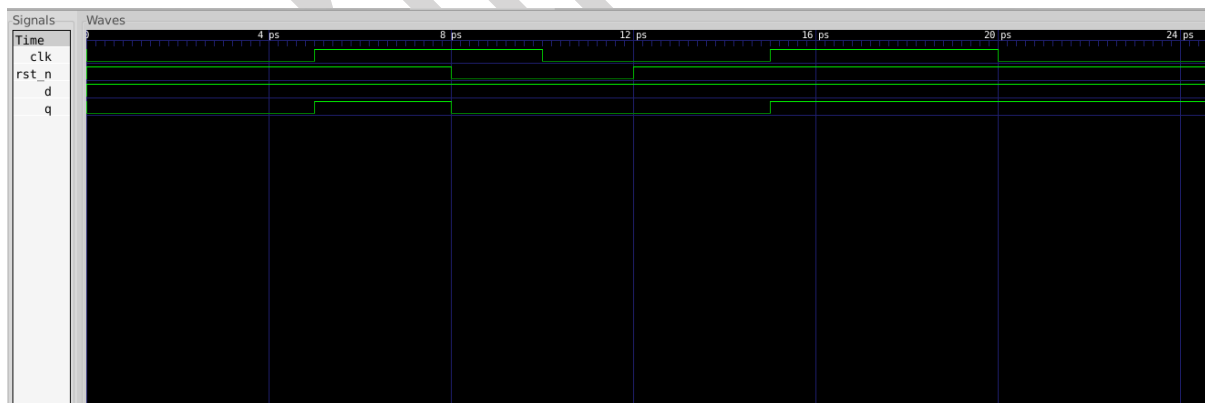
# Run Verilator to generate simulation files
verilator --binary -j 0 -Wall async_reset_ff.v \
async_reset_tb.v --top async_reset_tb \
--timing --CFLAGS "-std=c++20" --trace
cd obj_dir || { echo "obj_dir not found"; exit 1; }
make -f Vasync_reset_tb.mk Vasync_reset_tb || { echo
"Compilation failed"; exit 1; }
./Vasync_reset_tb || { echo "Simulation failed"; exit 1; }
gtkwave async_reset.vcd
    
```

b) Make the script executable:

```
chmod +x run.sh
```

1.5 : Run the script to perform compilation, simulation, and open the waveform.

GTKWave Observation:



- Output q goes low immediately after rst_n = 0, regardless of clock.
- When rst_n is high again, normal operation resumes.
- Asynchronous resets override clock, are crucial for external or fast resets.

Lab -2: Synchronous Reset

Objective of this lab is to observe how reset only takes effect at clock edges, even if asserted earlier.

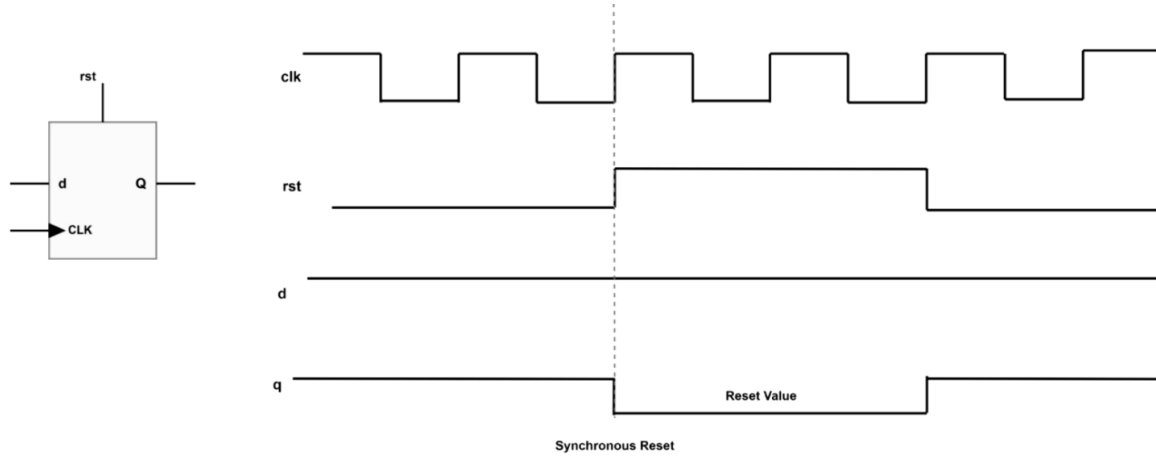


Fig: Synchronous reset

2.1 Navigate to synchronous_reset directory

2.2 Create the files for synchronous_reset: sync_reset_ff.v

```
module sync_reset_ff (
    input clk,
    input rst,        // Active-high reset
    input d,
    output reg q
);
    always @(posedge clk) begin
        if (rst)
            q <= 0;
        else
            q <= d;
        end
    end
endmodule
```

2.3 Create the file for testbench: sync_reset_tb.v

```
module sync_reset_tb;
    reg clk = 0, rst = 0, d = 0;
```

```
wire q;

sync_reset_ff uut (.clk(clk), .rst(rst), .d(d), .q(q));

always #5 clk = ~clk;

initial begin

    $dumpfile("sync_reset.vcd");

    $dumpvars(0, sync_reset_tb);

    d = 1; #3;

    rst = 1; #7;

    rst = 0; #20;

    $finish;

end

endmodule
```

2.4 : Simulation Script

a) Create “run.sh”

```
#!/bin/bash

# Run Verilator to generate simulation files
verilator --binary -j 0 -Wall sync_reset_ff.v \
sync_reset_tb.v --top sync_reset_tb \
--timing --CFLAGS "-std=c++20" --trace

# Navigate into the build directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }

# Compile the testbench
make -f Vsync_reset_tb.mk Vsync_reset_tb || { echo "Compilation
failed"; exit 1; }

# Run the simulation
./Vsync_reset_tb || { echo "Simulation failed"; exit 1; }

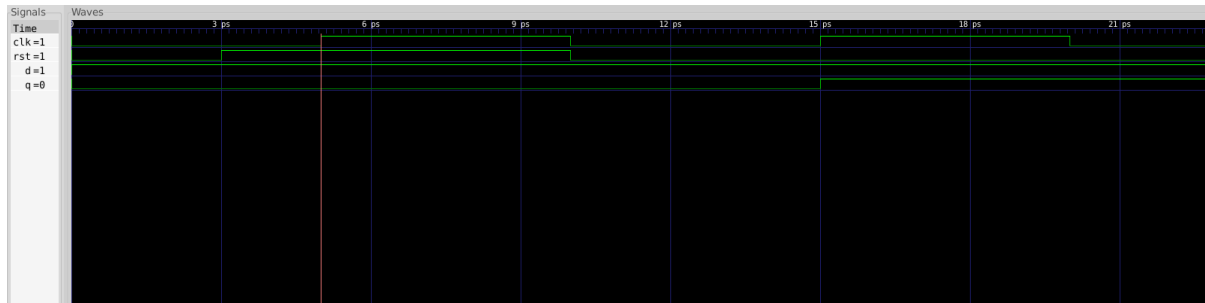
# Open the waveform in GTKWave
gtkwave sync_reset.vcd
```

b) Make the script executable:

```
chmod +x run.sh
```

2.5: Run the script to perform compilation, simulation, and open the waveform.

2.6 :GTKWave Observation:



- Output q resets only at the rising edge of clk when rst is asserted.
- Synchronous resets give predictable, synthesis-friendly circuit behavior.

Lab 3: Global Reset

Objective of this lab is to show how a single reset signal can reset multiple flip-flops or blocks.

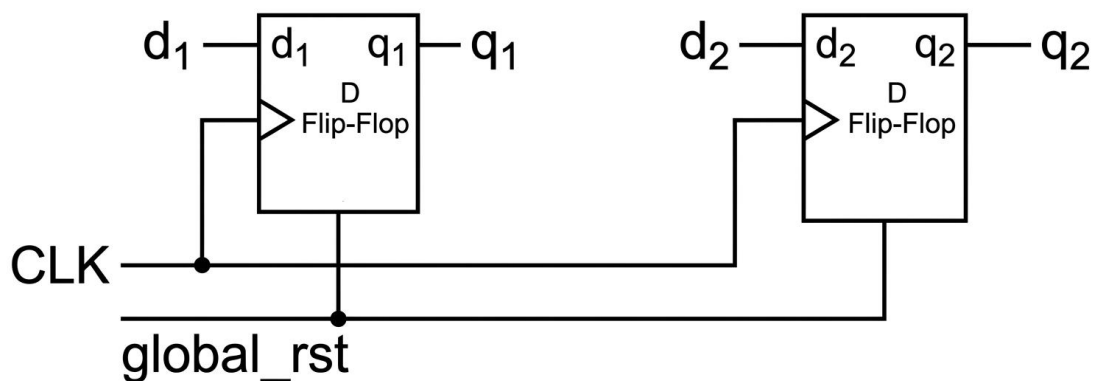


Fig: Global Reset

3.1 Navigate to global_reset directory

3.2 Create the files for global_reset : global_reset.v

```
module global_reset (
    input clk,
    input global_reset,
    input d1, d2,
    output reg q1, q2
);
    always @(posedge clk) begin
```

```
    if (global_rst) begin
        q1 <= 0;
        q2 <= 0;
    end else begin
        q1 <= d1;
        q2 <= d2;
    end
end
endmodule
```

3.3 :Create a file for global reset testbench : global_reset_tb.v

```
module global_reset_tb;
    reg clk = 0, global_rst = 0;
    reg d1 = 1, d2 = 1;
    wire q1, q2;

    global_reset uut (.clk(clk), .global_rst(global_rst),
        .d1(d1), .d2(d2), .q1(q1), .q2(q2));

    always #5 clk = ~clk;
    initial begin
        $dumpfile("global_reset.vcd");
        $dumpvars(0, global_reset_tb);
        #10 global_rst = 1;
        #10 global_rst = 0;
        #20 $finish;
    end
endmodule
```

3.4: Simulation Script

a) Create “run.sh”

```
#!/bin/bash

# Run Verilator to generate simulation files
verilator --binary -j 0 -Wall global_reset.v \
global_reset_tb.v --top global_reset_tb \
```

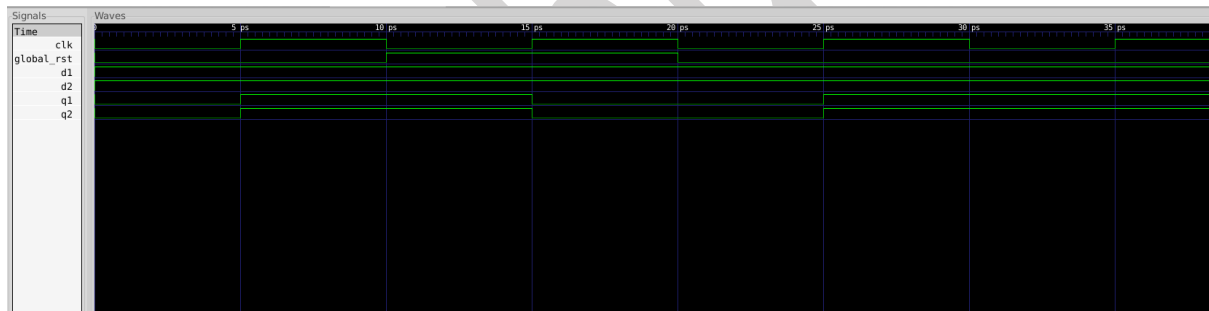
```
--timing --CFLAGS "-std=c++20" --trace  
# Navigate into the object directory  
cd obj_dir || { echo "obj_dir not found"; exit 1; }  
# Compile the testbench  
make -f Vglobal_reset_tb.mk Vglobal_reset_tb || { echo  
"Compilation failed"; exit 1; }  
# Run the simulation  
./Vglobal_reset_tb || { echo "Simulation failed"; exit 1; }  
# Open the waveform in GTKWave  
gtkwave global_reset.vcd
```

b) Make the script executable:

```
chmod +x run.sh
```

3.5: Run the script to perform compilation, simulation, and open the waveform.

3.6 : GTKWave Observation:



- Both q1 and q2 are reset to 0 together at the next active clock edge while global_rst is high.
- System-wide global resets are used for SoC start-up, boot, or processor resets.

Lab 4: Domain-Specific Reset

Objective of this is to understand how subsystems use independent clocks and resets.

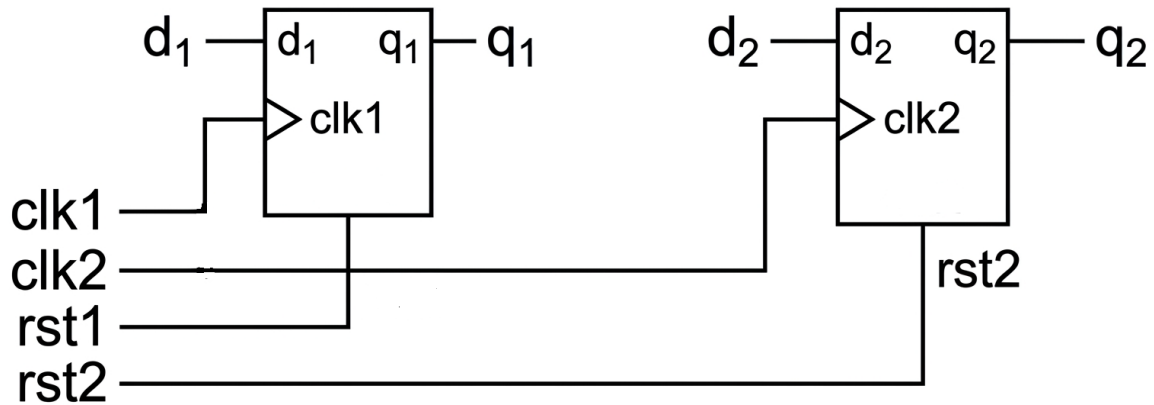


Fig : Domain specific reset

4.1 Navigate to domain_specific_reset directory

4.2 Create the file for domain_specific_reset : domain_reset.v

```
module domain_reset
    input clk1, clk2,
    input rst1, rst2,
    input d1, d2,
    output reg q1, q2
)
always @(posedge clk1) begin
    if (rst1)
        q1 <= 0;
    else
        q1 <= d1;
    end
always @(posedge clk2) begin
    if (rst2)
        q2 <= 0;
    else
        q2 <= d2;
    end
end
```

end

endmodule

4.3 Create the file for domain_testbench : domain_reset_tb.v

```
module domain_reset_tb;

    reg clk1 = 0, clk2 = 0;
    reg rst1 = 0, rst2 = 0;
    reg d1 = 1, d2 = 1;
    wire q1, q2;

    domain_reset uut (.clk1(clk1), .clk2(clk2), .rst1(rst1),
    .rst2(rst2), .d1(d1), .d2(d2), .q1(q1), .q2(q2));

    always #4 clk1 = ~clk1;
    always #6 clk2 = ~clk2;
    initial begin
        $dumpfile("domain_reset.vcd");
        $dumpvars(0, domain_reset_tb);
        #10 rst1 = 1; #8; rst1 = 0;
        #15 rst2 = 1; #10; rst2 = 0;
        #30 $finish;
    end
end
endmodule
```

4.4: Simulation Script

a) Create "run.sh"

```
#!/bin/bash

#Run Verilator to generate simulation files
verilator --binary -j 0 -Wall domain_reset.v \
domain_reset_tb.v --top domain_reset_tb --timing --CFLAGS \
"-std=c++20" --trace

#Navigate into the object directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }

#Compile the testbench
```

```
make -f Vdomain_reset_tb.mk Vdomain_reset_tb || { echo  
"Compilation failed"; exit 1; }
```

#Run the simulation

```
./Vdomain_reset_tb || { echo "Simulation failed"; exit 1; }
```

Step 5: View the waveform in GTKWave

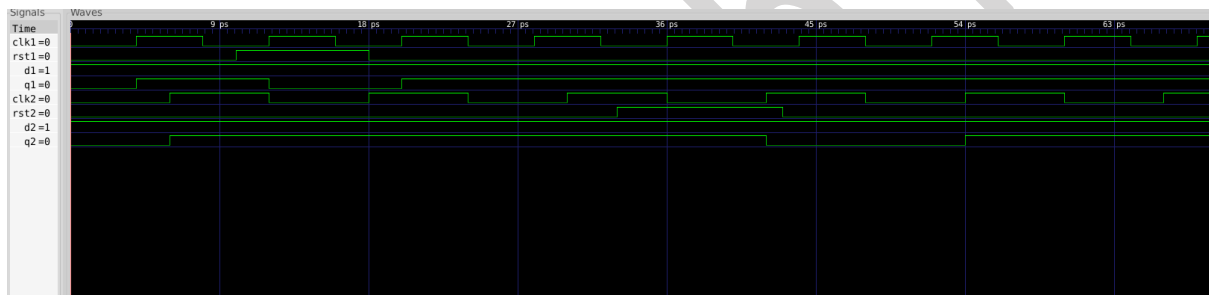
```
gtkwave domain_reset.vcd
```

b) Make the script executable:

```
chmod +x run.sh
```

4.5: Run the script to perform compilation, simulation, and open the waveform.

4.6 :GTKWave Observation:



- q1 and q2 are reset by their own reset signals, independent of each other, at their respective clocks.
- Subsystem/IP blocks can be independently reset, essential for complex SoCs.

Summary Table

Reset Type	Trigger	Clock Dependent	Scope	Typical Use Case
Asynchronous	Signal edge	No	Module-level	External resets, fast initialization
Synchronous	Clock edge	Yes	Module-level	ASICs, synchronous reset trees
Global Reset	Clock edge	Yes	System-wide	Chip-wide initialization

Reset Type	Trigger	Clock Dependent	Scope	Typical Use Case
Domain-Specific	Clock edge	Yes	Subsystem/IP	SoCs with multiple functional domains

Conclusion:

Understanding and selecting the right type of reset is critical for robust RTL design.

- **Asynchronous resets** provide immediate control, suitable for external or emergency resets.
- **Synchronous resets** offer predictable and synthesis-friendly behavior, ideal for modern ASIC/FPGA flows.
- **Global resets** ensure uniform system startup.
- **Domain-specific resets** allow flexible and granular control in complex SoCs.